

UNIVERSIDADE DE CAXIAS DO SUL

GERÊNCIA DE CONFIGURAÇÃO

GUILHERME BAREA

GUILHERME MOLON

GUILHERME RECH

LUIZ PEREIRA

IA GENERATIVA E QUALIDADE DE CÓDIGO

CAXIAS DO SUL

O código escrito com ajuda de IA é melhor ou pior do que o código escrito sem ela?

A conclusão de que os códigos ficam "melhores" é verdadeira na superfície e no curto prazo (nível local), mas falsa na arquitetura e no longo prazo (nível sistêmico).

Por um lado (Melhor no nível micro e local — Dados Numéricos):

Taxa de acerto funcional em tarefas fáceis: O modelo obteve uma taxa de sucesso de 89,0% em Python e 86,0% em Java no benchmark de 501 tarefas de nível fácil do LeetCode.

Redução de violações de regras de estilo: No estudo com estudantes de Java, o grupo com IA reduziu as violações com significância estatística ($p < 0,005$).

A mediana da regra de comprimento de linha (LineLength) caiu de 50,3 (grupo de controle) para 16,65 (grupo com IA).

A mediana da regra de parâmetros finais (FinalParameters) caiu de 28,57 para 16,23.

No exercício de Coleções, o total de violações registradas caiu de 4.632 para 2.766.

Métricas de complexidade do código:

A complexidade ciclomática teve mediana geral de 7,27 no grupo assistido por IA contra 9,17 no grupo de controle ($p < 0,005$), registrando 5,91 contra 8,67 no exercício de Programação Orientada a Objetos.

A complexidade cognitiva registrou mediana geral de 1,34 no grupo com IA contra 1,50 no controle ($p < 0,005$).

Auto-reparo guiado: A inclusão de relatórios de análise estática permitiu ao modelo corrigir mais de 60% dos problemas de estilo e manutenibilidade em código Python.

Por outro lado (Pior no nível macro e sistêmico):

Em repositórios de produção (com 24,1% de commits via IA em 2026), observam-se quedas estruturais: a refatoração por movimentação de linhas caiu 70% (de 12,7% para 3,8% das alterações), a manutenção de código legado caiu 74% (de 1,71% para 0,46%) e a conectividade entre funções reduziu 35% (de 343 para 223 chamadas por mil linhas).

Paralelamente, a duplicação de blocos de código (5+ linhas) subiu 81% (de 40,3 para 73,0 linhas por mil alteradas), o hábito de copiar e colar dentro dos commits cresceu 41% (chegando a 15,7% do código alterado) e a densidade de estruturas para mascarar erros aumentou 47% (de 16,4 para 24,1 por mil linhas).

Em suma, ignorar a qualidade na escrita para obter velocidade imediata transfere um custo desproporcional para os desenvolvedores que precisarão manter o sistema nos anos seguintes

O que é código de qualidade? Todo mundo concorda com a mesma definição?

Não existe um consenso universal sobre a definição de código de qualidade.

Pesquisas que investigam as percepções de alunos, educadores e desenvolvedores profissionais revelam que não há uma definição única compartilhada entre esses grupos. Enquanto algumas pessoas priorizam desempenho e estrutura, outras enfatizam a concisão ou a clareza, sendo a legibilidade o indicador de qualidade citado com maior frequência. Além disso, análises na engenharia de software apontam que "qualidade de código" é um termo amplo sem uma definição consensual, pois tentar estabelecer uma regra universal de "código perfeito" para qualquer cenário esbarra na complexidade dos casos específicos de cada projeto. Por conta dessa dificuldade de criar uma definição positiva única, a literatura científica e as ferramentas automatizadas costumam operacionalizar e medir a qualidade de código acumulando exclusões de antipadrões e defeitos (ou seja, definindo o que *não* é código de qualidade). As pesquisas estruturam essa avaliação em quatro pilares principais:

Conformidade de Estilo e Layout:

Envolve a organização visual das instruções, nomenclatura e adesão rigorosa a convenções de codificação estabelecidas (como as convenções da Oracle para Java), sendo auditada por ferramentas de análise estática como Checkstyle, PMD, Pylint e Flake8.

Corretude Funcional e Confiabilidade:

Mede a capacidade do programa de compilar, executar sem falhas em tempo de execução (como estouro de índices ou incompatibilidade de tipos) e produzir saídas corretas diante de suítes de teste.

Complexidade e Compreensibilidade:

Avaliada por métricas quantitativas como a complexidade ciclomática (número de caminhos de execução independentes) e a complexidade cognitiva (o grau de esforço mental exigido para entender o aninhamento e o fluxo do código).

Manutenibilidade e Saúde Arquitetural:

Examina o impacto do código no longo prazo, considerando a ausência de "code smells" (como variáveis declaradas e nunca utilizadas), taxas de refatoração e reutilização de módulos, conectividade entre funções e a ausência de antipadrões como duplicação de blocos ou mascaramento de exceções.

Em suma, a qualidade de código não é um conceito estático ou estritamente homogêneo, mas sim uma síntese de conformidade sintática, eficiência funcional, facilidade de leitura e manutenibilidade estrutural ao longo do tempo.

Que partes da qualidade uma ferramenta automática consegue medir? E que partes só um humano percebe?

Com base nas evidências apresentadas nos três artigos, a avaliação da qualidade de código divide-se entre o que as ferramentas automatizadas conseguem quantificar analiticamente e o que depende exclusivamente da visão contextual e crítica do ser humano:

O que uma Ferramenta Automática Consegue Medir?

As ferramentas automatizadas operam em três níveis: análise estática de regras locais, métricas quantitativas de complexidade e detecção de sinais de risco estrutural em escala de repositório.

Conformidade de Estilo e Convenções de Código:

Analísadores estáticos como Checkstyle, PMD, Pylint e Flake8 verificam objetivamente se o código segue padrões sintáticos e convenções formais de codificação (como as convenções da Oracle para Java). Eles medem:

Comprimento excessivo de linhas (LineLength).

Parâmetros não marcados como final (FinalParameters).

Variáveis locais que ocultam campos da classe (HiddenField).

Ausência de cláusulas default em instruções switch (MissingSwitchDefault).

Antipadrões locais (*code smells*), como variáveis declaradas e nunca utilizadas (UnusedVariable), reatribuição de parâmetros ou modificadores redundantes.

Métricas Quantitativas de Complexidade:

Complexidade Ciclômática (medida via Checkstyle/McCabe): conta o número exato de caminhos de execução independentes e bifurcações de fluxo dentro de um método.

Complexidade Cognitiva (medida via SonarQube): calcula o esforço mental exigido para compreender o código avaliando a profundidade de aninhamento de estruturas de controle e operadores lógicos.

Complexidade NPath (medida via PMD): quantifica o número de caminhos de execução acíclicos em uma função.

Verificação Funcional e Erros de Execução: Suítes de testes automatizados e compiladores identificam erros de compilação, exceções em tempo de execução (como NullPointerException, divisão por zero ou acesso a índices inválidos) e saídas incorretas (*Wrong Outputs*) em relação aos resultados esperados.

Sinais Estruturais em Escala de Repositório:

Ferramentas de análise de histórico de commits (como a GitClear) monitoram a evolução estrutural em grandes massas de código.

Duplicação de blocos: identificação de sequências de 5 ou mais linhas consecutivas idênticas repetidas no repositório.

Taxa de *Copy/Paste* versus Movimentação (*Move*): percentual de linhas copiadas versus linhas movidas para refatoração dentro dos commits.

Densidade de Mascaramento de Erros (*Obfuscation Rate*): contagem automatizada de blocos catch/rescue, operadores de navegação segura (&.) e métodos *stub/mock* em testes.

Atrito e Descarte (*Churn* e *Throwaway Code*): percentual de código reescrito ou descartado em até 1 semana (*throwaway*) ou 2 semanas (*churn*).

Conectividade de Funções e Atualizações Legadas: frequência com que novos métodos chamam funções de outros arquivos (*Function Connectivity*) e o percentual de alterações em códigos com mais de 12 meses.

O que Só um Humano Percebe

Ferramentas automatizadas conseguem contar linhas, erros e estruturas, mas não possuem compreensão semântica, intencionalidade ou visão arquitetural. A avaliação humana é indispensável em aspectos como:

Adequação Arquitetural e Reuso Intencional: A ferramenta mede que o código não possui erros sintáticos, mas não sabe se aquele código deveria existir. Só um desenvolvedor humano percebe quando uma nova funcionalidade gerada por IA está "reinventando a roda" em um arquivo isolado em vez de consolidar e estender abstrações já existentes no sistema.

Intenção do Negócio e Semântica Profunda: Uma IA pode produzir um código que passa em todos os testes automatizados e apresenta baixíssima complexidade ciclomática, mas que implementa uma regra de negócio conceitualmente errada ou calcula um resultado equivocado por erro de interpretação. O analisador automático valida a execução, mas só o humano avalia a adequação ao contexto real.

Legitimidade do Tratamento de Exceções (Mascaramento vs. Tratamento Real): Ferramentas conseguem contar quantos blocos catch/rescue foram inseridos, mas não julgam a intenção. Só o humano consegue distinguir se um tratamento de erro é uma resposta válida a uma falha real de infraestrutura ou se foi inserido apenas para silenciar exceções, fazer os testes passarem e varrer o bug para debaixo do tapete.

Qualidade e Validade das Suítes de Testes: Relatórios automatizados indicam porcentagens de cobertura de testes, mas só um humano percebe quando um teste unitário foi "alucinadamente" criado pela IA apenas para justificar uma estrutura desnecessária ou quando o teste valida cenários implausíveis enquanto ignora os fluxos reais de uso.

Decisão Estratégica de Trade-Offs e Momento de Refatorar: Saber quando pausar a entrega de novas funcionalidades para reestruturar o sistema, aceitando um risco controlado no curto prazo para garantir a simplicidade e a manutenibilidade do software no longo prazo, é uma decisão de engenharia exclusivamente humana.

Qualidade importa mais no momento de escrever o código ou anos depois, quando alguém precisa mantê-lo?

A qualidade de código importa desproporcionalmente mais anos depois, durante a fase de manutenção, embora a sustentabilidade dessa manutenção dependa de decisões arquiteturais tomadas no momento da escrita inicial.

O Momento da Escrita ("Dia 1")

A IA reduziu drasticamente o custo e o esforço necessário para gerar novas linhas de código. No momento da escrita inicial e em tarefas isoladas, a qualidade superficial do código assistido por IA frequentemente impressiona:

- Entrega alta conformidade sintática e de estilo.
- Reduz métricas de complexidade ciclomática e cognitiva.
- Resolve o problema imediato do ticket de forma rápida.

Contudo, a IA tende a gerar "código atômico": uma solução focada no caminho feliz e no teste pontual, sem realizar as reestruturações ao redor que um desenvolvedor humano sênior faria de forma intuitiva. Como criar novos arquivos e adicionar linhas tornou-se fácil, a escrita inicial prioriza a velocidade imediata em detrimento da estrutura global.

A Fase de Manutenção ("Anos Depois"):

É no longo prazo, quando a equipe precisa manter, estender e corrigir o sistema, que o "boleto" da falta de qualidade arquitetural é cobrado:

Taxa de Propagação de Defectos: A duplicação de blocos de código (sequências de 5+ linhas repetidas) subiu 81% (de 40,3 para 73,0 linhas por mil alteradas). Estudos mostram que 57% dos clones de código co-alterados estão envolvidos em bugs, e entre 17% e 33% das duplicatas contêm falhas propagadas. Quando uma alteração é necessária anos depois, o mantenedor herda a obrigação de encontrar e atualizar todas as réplicas espalhadas — as que forem esquecidas transformam-se em bugs em produção.

Colapso da Refatoração:

A movimentação de linhas para consolidar o sistema despencou 70% (de 12,7% das alterações para 3,8%). Sem refatoração periódica, a base de código deixa de evoluir de forma coesa e decai em uma coleção de módulos isolados em "eterna versão 1" (*perpetual V1*) que ninguém consegue compreender de forma integrada.

Calcificação do Código Legado:

As alterações em trechos de código com mais de 12 meses caíram 74% (de 1,71% para 0,46%). Os componentes mais antigos e fundamentais da aplicação ficam congelados e negligenciados até quebrarem inesperadamente.

Camuflagem de Falhas de Sistema:

A densidade de estruturas para mascarar erros (como blocos catch/rescue abrangentes e navegadores seguros) aumentou 47% (de 16,4 para 24,1 por mil linhas). Isso "varre o erro para debaixo do tapete" para fazer os testes passarem na escrita, mas dificulta enormemente a identificação das causas raízes de falhas anos depois.

Conclusão

A qualidade do código importa desproporcionalmente mais anos depois, durante a fase de manutenção, do que no momento da escrita inicial. Usar a IA para acelerar a geração de linhas sem uma rigorosa supervisão humana e sem políticas ativas de refatoração apenas gera uma limpeza superficial no curto prazo, transferindo um desproporcional e pesado débito técnico para quem precisará manter e evoluir o sistema no futuro.

Referências Bibliográficas

Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues

- Autores: Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le e David Lo.
- Periódico: *ACM Transactions on Software Engineering and Methodology* (TOSEM), Vol. 33, N.º 5, Artigo 116 (junho de 2024).

Does ChatGPT Help Novice Programmers Write Better Code? Results From Static Code Analysis

- Autores: Philipp Haindl e Gerald Weinberger.
- Periódico: *IEEE Access*, Vol. 12, págs. 116933–116948 (agosto de 2024).

The Maintainability Gap: AI Code Quality in 2026

- Autor: William Harding.
- Instituição / Publicação: *GitClear and Alloy.dev Research* (Patrocinado por GitClear e GitKraken Research), publicado em junho de 2026.